Listing 24.16 Setting the `CanEditRecipe` property in the View.cshtml Razor Page

```
public class ViewModel : PageModel
{
    public Recipe Recipe { get; set; }
    public bool CanEditRecipe { get; set; }                          ❶

    private readonly RecipeService _service;
    private readonly IAuthorizationService _authService;
    public ViewModel(
        RecipeService service,
        IAuthorizationService authService)
    {
        _service = service;
        _authService = authService;
    }

    public async Task<IActionResult> OnGetAsync(int id)
    {
        Recipe = _service.GetRecipe(id);                             ❷
        AuthorizationResult isAuthorised = await _authService        ❸
            .AuthorizeAsync(User, recipe, "CanManageRecipe");        ❸
        CanEditRecipe = isAuthorised.Succeeded;                      ❹
        return Page();
    }
}
```

❶ The CanEditRecipe property will be used to control whether the Edit button is rendered.

❷ Loads the Recipe resource for use with IAuthorizationService

❸ Verifies whether the user is authorized to edit the Recipe

❹ Sets the CanEditRecipe property on the PageModel as appropriate

Instead of blocking execution of the Razor Page (as you did previously in the Edit.cshtml page handler), use the result of the call to `AuthorizeAsync` to set the `CanEditRecipe` value on the `PageModel`. You can then make a simple change to the View.chstml Razor template, adding an `if` clause around the rendering of the Edit link:

```
@if(Model.CanEditRecipe)
{
```